

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC



BÁO CÁO BÀI TẬP NHÓM
MÔN: CÁC HỆ THỐNG PHÂN TÁN
HỆ THỐNG CHUYỂN TIỀN LIÊN NGÂN HÀNG QUA
NAPAS SỬ DỤNG KIẾN TRÚC MICROSERVICES

Giảng viên hướng dẫn : TS. Kim Ngọc Bách

Lớp : M25CQHT01-B

Nhóm 2

Nguyễn Thị Bích Hà : B25CHHT017

Phạm Huy Hiến : B25CHHT019

Phạm Văn Tùng : B25CHHT064

Hà Nội - 2025

MỤC LỤC

PHẦN I: PHÂN TÍCH YÊU CẦU	4
1. Bài Toán Đặt Ra	4
2. Yêu Cầu Chức Năng	4
2.1. Xác Thực và Phân Quyền	4
2.2. Quản Lý Tài Khoản	4
2.3. Chuyển Tiền Nội Bộ	5
2.4. Chuyển Tiền Liên Ngân Hàng	5
2.5. Lịch Sử Giao Dịch	6
2.6. Đối Soát (NAPAS)	6
2.7. Thống Kê	6
3. Yêu Cầu Phi Chức Năng	7
3.1. Hiệu Năng	7
3.2. Khả Năng Mở Rộng	7
3.3. Độ Tin Cậy	7
3.4. Bảo Mật (Trong Phạm Vi Demo)	7
3.5. Khả Năng Vận Hành	7
PHẦN II: LIỆT KÊ TÍNH NĂNG	8
1. Tính Năng Đã Triển Khai	8
1.1. Dịch Vụ Xác Thực (Auth Service)	8
1.2. Dịch Vụ Ngân Hàng (Bank A/B/C Services)	8
1.3. Dịch Vụ NAPAS (NAPAS Service)	9
1.4. API Gateway (Ocelot)	9
1.5. Frontend	10
1.6. Message Queue (RabbitMQ)	10
2. Bảng Tổng Hợp Tính Năng	12
2.1. Tổng Quan Hệ Thống	12
2.2. Chi Tiết Từng Service	12
2.3. Phân Loại Theo Chức Năng	14
2.4. Communication Patterns	15
2.5. Database Schema Summary	16
2.6. Technology Stack Summary	16
2.7. Port Mapping	17
2.8. Demo Data Summary	18

PHẦN III: THIẾT KẾ PHẦN MỀM	19
1. Thiết Kế Tổng Thể	19
1.1. Kiến Trúc Layers	19
1.2. Design Patterns Sử Dụng	21
2. Thiết Kế Chi Tiết	22
2.1. Entity Relationship Diagrams (ERD)	22
2.2. Sequence Diagrams	26
2.3. Class Diagrams	28
3. Thiết Kế API	29
3.1. RESTful API Principles	29
3.2. API Endpoints Summary	29
3.2.1. Auth Service (Port 5001)	29
3.2.2. Bank Services (Bank A: 5003, Bank B: 5004, Bank C: 5005)	30
3.2.3. NAPAS Service (Port 5002)	31
3.2.4. API Gateway (Ocelot - Port 5000)	32
4. Thiết Kế Message Queue	33
4.1. Queue Configuration	33
4.2. Message Flow	33
4.3. Error Handling	33
5. Thiết Kế Security	34
5.1. Authentication (JWT)	34
5.2. Authorization	34
5.3. Data Protection	34
6. Thiết Kế Deployment	34
6.1. Containerization	34
6.2. Orchestration	34
6.3. Network	34
6.4. Volumes	34

PHẦN I: PHÂN TÍCH YÊU CẦU

1. Bài Toán Đặt Ra

Hiện nay, hệ thống thanh toán liên ngân hàng tại Việt Nam được vận hành bởi NAPAS (National Payment Corporation of Vietnam - Tổng Công ty Thanh toán Quốc gia Việt Nam). Hệ thống này cho phép khách hàng của các ngân hàng khác nhau có thể chuyển tiền cho nhau một cách nhanh chóng và an toàn.

Thách thức:

- Các ngân hàng hoạt động độc lập với hệ thống riêng biệt
- Cần một trung gian đáng tin cậy để định tuyến và xử lý giao dịch
- Đảm bảo tính nhất quán dữ liệu trong môi trường phân tán
- Xử lý bất đồng bộ để tối ưu hiệu suất
- Ghi nhận đầy đủ để đối soát và kiểm toán

2. Yêu Cầu Chức Năng

Nhóm đã tự đặt ra các yêu cầu chức năng sau cho hệ thống:

2.1. Xác Thực và Phân Quyền

- **UC-01:** Đăng nhập người dùng
 - Actor: Người dùng ngân hàng
 - Mô tả: Người dùng đăng nhập vào hệ thống của ngân hàng để sử dụng dịch vụ
 - Input: Username, Password
 - Output: JWT Token
 - Business Rules:
 - Thông tin đăng nhập phải hợp lệ
 - Token có thời hạn sử dụng
 - Mỗi người dùng thuộc về một ngân hàng cụ thể

2.2. Quản Lý Tài Khoản

- **UC-02:** Xem thông tin tài khoản
 - Actor: Người dùng ngân hàng
 - Mô tả: Người dùng xem danh sách tài khoản và số dư
 - Input: JWT Token
 - Output: Danh sách tài khoản với số dư hiện tại

- Business Rules:
 - Chỉ hiển thị tài khoản thuộc ngân hàng của người dùng
 - Số dư phải được cập nhật real-time

2.3. Chuyển Tiền Nội Bộ

- **UC-03:** Chuyển tiền trong cùng ngân hàng
 - Actor: Người dùng ngân hàng
 - Mô tả: Chuyển tiền giữa hai tài khoản trong cùng ngân hàng
 - Input: Tài khoản nguồn, tài khoản đích, số tiền, mô tả
 - Output: Kết quả giao dịch (SUCCESS/FAILED)
 - Business Rules:
 - Tài khoản nguồn và đích phải tồn tại
 - Số dư tài khoản nguồn $\geq$ số tiền chuyển
 - Giao dịch phải atomic (all-or-nothing)
 - Cập nhật số dư tức thì
 - Lưu lại lịch sử giao dịch

2.4. Chuyển Tiền Liên Ngân Hàng

- **UC-04:** Chuyển tiền qua NAPAS
 - Actor: Người dùng ngân hàng
 - Mô tả: Chuyển tiền đến tài khoản ở ngân hàng khác qua NAPAS
 - Input: Tài khoản nguồn, ngân hàng đích, tài khoản đích, số tiền, mô tả
 - Output: Transaction ID, trạng thái PENDING
 - Business Rules:
 - Trừ tiền tài khoản nguồn ngay lập tức
 - Gửi yêu cầu đến NAPAS qua message queue
 - NAPAS định tuyến đến ngân hàng đích
 - Ngân hàng đích xử lý và trả kết quả
 - Trạng thái cuối cùng: SUCCESS/FAILED
 - Thời gian xử lý: 2-3 giây

2.5. Lịch Sử Giao Dịch

- **UC-05:** Xem lịch sử giao dịch
 - Actor: Người dùng ngân hàng
 - Mô tả: Xem tất cả giao dịch của tài khoản
 - Input: Số tài khoản
 - Output: Danh sách giao dịch với chi tiết
 - Business Rules:
 - Hiển thị cả giao dịch đến và đi
 - Phân biệt giao dịch nội bộ và liên ngân hàng
 - Hiển thị trạng thái và thời gian

2.6. Đối Soát (NAPAS)

- **UC-06:** Tạo bản ghi đối soát
 - Actor: NAPAS Service
 - Mô tả: NAPAS ghi nhận và lưu trữ thông tin giao dịch để đối soát
 - Input: Thông tin giao dịch hoàn thành
 - Output: Bản ghi đối soát
 - Business Rules:
 - Mỗi giao dịch liên ngân hàng có bản ghi đối soát
 - Lưu trữ vĩnh viễn
 - Dùng để thanh toán giữa các ngân hàng

2.7. Thống Kê

- **UC-07:** Xem thống kê giao dịch
 - Actor: NAPAS Admin
 - Mô tả: Xem thống kê tổng quan về giao dịch
 - Output: Tổng giao dịch, tỷ lệ thành công, tổng giá trị
 - Business Rules:
 - Tính toán real-time
 - Phân loại theo trạng thái
 - Hiển thị xu hướng

3. Yêu Cầu Phi Chức Năng

3.1. Hiệu Năng

- Thời gian phản hồi API < 200ms (không bao gồm xử lý bất đồng bộ)
- Chuyển tiền nội bộ hoàn thành < 100ms
- Chuyển tiền liên ngân hàng hoàn thành < 5 giây
- Hệ thống có thể xử lý 100+ giao dịch đồng thời

3.2. Khả Năng Mở Rộng

- Hỗ trợ thêm ngân hàng mới mà không ảnh hưởng hệ thống hiện tại
- Có thể scale horizontal (thêm instances)
- Database có thể tách riêng và scale độc lập

3.3. Độ Tin Cậy

- Message queue đảm bảo không mất tin nhắn (durable queues)
- Giao dịch database đảm bảo tính ACID
- Có cơ chế retry khi gặp lỗi tạm thời
- Log đầy đủ để debug

3.4. Bảo Mật (Trong Phạm Vi Demo)

- Xác thực dựa trên JWT
- Mỗi ngân hàng có database riêng biệt
- Không có truy cập trực tiếp giữa các database

3.5. Khả Năng Vận Hành

- Triển khai dễ dàng với Docker Compose
- Có health check endpoints
- Log tập trung
- Có thể monitor qua RabbitMQ Management Console

PHẦN II: LIỆT KÊ TÍNH NĂNG

1. Tính Năng Đã Triển Khai

1.1. Dịch Vụ Xác Thực (Auth Service)

✓ **Chức năng:**

- Đăng nhập với username/password
- Tạo JWT token
- Lưu trữ thông tin người dùng
- Quản lý roles (User/Admin)

✓ **API Endpoints:**

- POST /api/auth/login - Đăng nhập
- GET /api/auth/users - Danh sách người dùng
- POST /api/auth/admin/create-tables - Tạo bảng
- POST /api/auth/admin/seed - Seed dữ liệu
- POST /api/auth/admin/reset - Reset dữ liệu

1.2. Dịch Vụ Ngân Hàng (Bank A/B/C Services)

✓ **Chức năng:**

- Quản lý tài khoản
- Xem số dư
- Chuyển tiền nội bộ (tức thì)
- Khởi tạo chuyển tiền liên ngân hàng
- Nhận chuyển tiền từ ngân hàng khác
- Lịch sử giao dịch
- Cập nhật real-time qua SignalR

✓ **API Endpoints:**

- GET /api/accounts - Danh sách tài khoản
- GET /api/accounts/{accountNumber} - Chi tiết tài khoản
- GET /api/accounts/{accountNumber}/transactions - Lịch sử giao dịch
- POST /api/transfer/internal - Chuyển tiền nội bộ
- POST /api/transfer/interbank - Chuyển tiền liên ngân hàng
- POST /api/admin/seed - Seed dữ liệu

- POST /api/admin/reset - Reset dữ liệu

✓ **SignalR Hub:**

- /balanceHub - WebSocket endpoint
- SubscribeToAccount(accountNumber) - Đăng ký nhận cập nhật
- UnsubscribeFromAccount(accountNumber) - Hủy đăng ký
- Events: BalanceUpdated, TransactionReceived, TransactionCompleted

1.3. Dịch Vụ NAPAS (NAPAS Service)

✓ **Chức năng:**

- Nhận yêu cầu chuyển tiền từ các ngân hàng
- Định tuyến đến ngân hàng đích
- Nhận kết quả và cập nhật trạng thái
- Tạo bản ghi đối soát
- Cung cấp thống kê

✓ **API Endpoints:**

- GET /api/transactions - Tất cả giao dịch
- GET /api/transactions/{transactionId} - Chi tiết giao dịch
- GET /api/reconciliation - Bản ghi đối soát
- GET /api/statistics - Thống kê tổng quan

✓ **Background Services:**

- TransferRouterService - Định tuyến giao dịch đến ngân hàng đích
- ResultProcessorService - Xử lý kết quả từ ngân hàng

1.4. API Gateway (Ocelot)

✓ **Chức năng:**

- Điểm vào duy nhất cho tất cả services
- Định tuyến request đến service phù hợp
- Xử lý CORS
- Aggregation (tương lai)

✓ Routes:

- /auth/* → Auth Service
- /banka/* → Bank A Service
- /bankb/* → Bank B Service
- /bankc/* → Bank C Service
- /napas/* → NAPAS Service

1.5. Frontend

✓ Tính năng:

- Đăng nhập
- Hiện thị danh sách tài khoản và số dư
- Form chuyển tiền (nội bộ và liên ngân hàng)
- Lịch sử giao dịch
- Thông báo real-time (SignalR)
- Multi-bank dashboard

✓ Công nghệ:

- HTML5 + CSS3 (Bootstrap 5)
- JavaScript ES6+
- Fetch API
- SignalR Client
- Nginx (để serve dashboard)

1.6. Message Queue (RabbitMQ)

✓ Chức năng:

- Trung gian giao tiếp bất đồng bộ giữa các services
- Đảm bảo reliable message delivery
- Hỗ trợ pattern publish/subscribe
- Message persistence (không mất dữ liệu khi restart)
- Decoupling giữa producer và consumer services

✓ **Queues:**

- transfer_napas - Nhận yêu cầu chuyển tiền từ các ngân hàng
- transfer_banka - Gửi yêu cầu đến Bank A Service
- transfer_bankb - Gửi yêu cầu đến Bank B Service
- transfer_bankc - Gửi yêu cầu đến Bank C Service
- transfer_result - Nhận kết quả xử lý từ các ngân hàng

✓ **Message Flow:**

- Bank Service → transfer_napas → NAPAS Service
- NAPAS Service → transfer_bank{x} → Destination Bank Service
- Destination Bank → transfer_result → NAPAS Service

✓ **Management Interface:**

- URL: http://localhost:15672
- Username/Password: guest/guest
- Giám sát queues, messages, connections
- Xem thống kê throughput và performance

✓ **Công nghệ:**

- RabbitMQ 3.x (Message Broker)
- AMQP Protocol (Port 5672)
- Management Plugin (Port 15672)
- Durable queues với persistent messages
- Manual acknowledgement để đảm bảo at-least-once delivery

2. Bảng Tổng Hợp Tính Năng

2.1. Tổng Quan Hệ Thống

Thành phần	Số lượng	Ghi chú
Microservices	6	Auth, NAPAS, Bank A, Bank B, Bank C, Gateway
Databases (PostgreSQL)	5	Database per service pattern
Message Queues	5	RabbitMQ queues
API Endpoints	30+	Tất cả services
Frontend Pages	1	Multi-bank dashboard

2.2. Chi Tiết Từng Service

Service	Chức năng chính	API Endpoints	Database	Port	Công nghệ
Auth Service	- Xác thực người dùng - Tạo JWT token - Quản lý users & roles	5 endpoints: - POST /api/auth/login - GET /api/auth/users - POST /api/auth/admin/create-tables - POST /api/auth/admin/seed - POST /api/auth/admin/reset	authdb (Port 5432)	5001	ASP.NET Core 8.0 Entity Framework Core PostgreSQL 15
Bank A Service	- Quản lý tài khoản - Chuyển tiền nội bộ - Chuyển tiền liên NH - Lịch sử giao dịch - Real-time updates	7 endpoints: - GET /api/accounts - GET /api/accounts/{id} - GET /api/accounts/{id}/transactions - POST /api/transfer/internal - POST /api/transfer/interbank - POST /api/admin/seed - POST /api/admin/reset	bankadb (Port 5434)	5003	ASP.NET Core 8.0 SignalR Hub RabbitMQ Client PostgreSQL 15

Service	Chức năng chính	API Endpoints	Database	Port	Công nghệ
Bank B Service	• Quản lý tài khoản • Chuyển tiền nội bộ • Chuyển tiền liên NH • Lịch sử giao dịch • Real-time updates	7 endpoints: (Same as Bank A)	bankbdb (Port 5435)	5004	ASP.NET Core 8.0 SignalR Hub RabbitMQ Client PostgreSQL 15
Bank C Service	• Quản lý tài khoản • Chuyển tiền nội bộ • Chuyển tiền liên NH • Lịch sử giao dịch • Real-time updates	7 endpoints: (Same as Bank A)	bankcdb (Port 5436)	5005	ASP.NET Core 8.0 SignalR Hub RabbitMQ Client PostgreSQL 15
NAPAS Service	• Định tuyến giao dịch • Lưu trữ bản ghi • Đối soát (reconciliation) • Thống kê tổng quan	4 endpoints: - GET /api/transactions - GET /api/transactions/{id} - GET /api/reconciliation - GET /api/statistics	napasdb (Port 5433)	5002	ASP.NET Core 8.0 RabbitMQ Client Background Services PostgreSQL 15
API Gateway	• Routing requests • Single entry point • CORS handling • Load balancing	-	Không có (Stateless)	5000	Ocelot ASP.NET Core 8.0
Frontend	• Giao diện đăng nhập	-	Không có	8080	HTML5 + CSS3

Service	Chức năng chính	API Endpoints	Database	Port	Công nghệ
	• Dashboard tài khoản • Form chuyển tiền • Lịch sử giao dịch • Real-time notifications				JavaScript ES6+ Bootstrap 5 SignalR Client Nginx
RabbitMQ	• Message broker • Async communication • Message persistence • Publish/Subscribe	5 queues: - transfer_napas - transfer_banka - transfer_bankb - transfer_bankc - transfer_result	Không có	5672 (AMQP) 15672 (UI)	RabbitMQ 3.x Management Plugin

2.3. Phân Loại Theo Chức Năng

Loại tính năng	Services liên quan	Thời gian xử lý	Pattern
Xác thực & Phân quyền	Auth Service	< 50ms	Synchronous (HTTP/REST)
Chuyển tiền nội bộ	Bank Services	< 100ms	Synchronous (ACID transaction)
Chuyển tiền liên ngân hàng	Bank Services → NAPAS → Bank Services	2-3 giây	Asynchronous (Message Queue)
Xem thông tin tài khoản	Bank Services	< 50ms	Synchronous (HTTP/REST)
Xem lịch sử giao dịch	Bank Services, NAPAS Service	< 100ms	Synchronous (HTTP/REST)
Thông báo real-time	Bank Services → Frontend	Instant	WebSocket (SignalR)

Loại tính năng	Services liên quan	Thời gian xử lý	Pattern
Đối soát & Thống kê	NAPAS Service	< 200ms	Synchronous (HTTP/REST)
Quản lý hệ thống	All Services	Varies	Synchronous (HTTP/REST)

2.4. Communication Patterns

Pattern	Use Case	Services	Protocol	Đặc điểm
Synchronous HTTP/REST	Queries, simple commands	All services	HTTP/HTTPS	• Request-Response • < 100ms • ACID transactions
Asynchronous Message Queue	Inter-bank transfers	Banks ↔ NAPAS	AMQP	• Non-blocking • 2-3 seconds • Eventual consistency
Real-time WebSocket	Balance updates, notifications	Banks → Frontend	WebSocket	• Bidirectional • Push notifications • Instant updates

2.5. Database Schema Summary

Database	Tables	Records (Demo)	Purpose
authdb	Users	4 users	Lưu thông tin xác thực
napasdb	NapasTransactions ReconciliationRecords	0 (generated)	Lưu giao dịch liên NH và đối soát
bankadb	Accounts Transactions	3 accounts 0 transactions	Tài khoản và giao dịch Bank A
bankbdb	Accounts Transactions	3 accounts 0 transactions	Tài khoản và giao dịch Bank B
bankcdb	Accounts Transactions	3 accounts 0 transactions	Tài khoản và giao dịch Bank C

2.6. Technology Stack Summary

Layer	Technology	Version	Purpose
Backend Services	ASP.NET Core	8.0	RESTful APIs, business logic
Database	PostgreSQL	15	Data persistence
Message Queue	RabbitMQ	3.x	Async communication
ORM	Entity Framework Core	8.0	Database access
Real-time	SignalR	8.0	WebSocket communication
API Gateway	Ocelot	Latest	API routing
Frontend	HTML/CSS/JS	ES6+	User interface
UI Framework	Bootstrap	5.x	Responsive design
Containerization	Docker	24.0+	Service deployment
Orchestration	Docker Compose	2.20+	Multi-container management

2.7. Port Mapping

Service/Component	Internal Port	External Port	Protocol
API Gateway	5000	5000	HTTP
Auth Service	5001	5001	HTTP
NAPAS Service	5002	5002	HTTP
Bank A Service	5003	5003	HTTP
Bank B Service	5004	5004	HTTP
Bank C Service	5005	5005	HTTP
Auth Database	5432	5432	PostgreSQL
NAPAS Database	5432	5433	PostgreSQL
Bank A Database	5432	5434	PostgreSQL
Bank B Database	5432	5435	PostgreSQL
Bank C Database	5432	5436	PostgreSQL
RabbitMQ AMQP	5672	5672	AMQP
RabbitMQ Management	15672	15672	HTTP
Frontend Dashboard	80	8080	HTTP

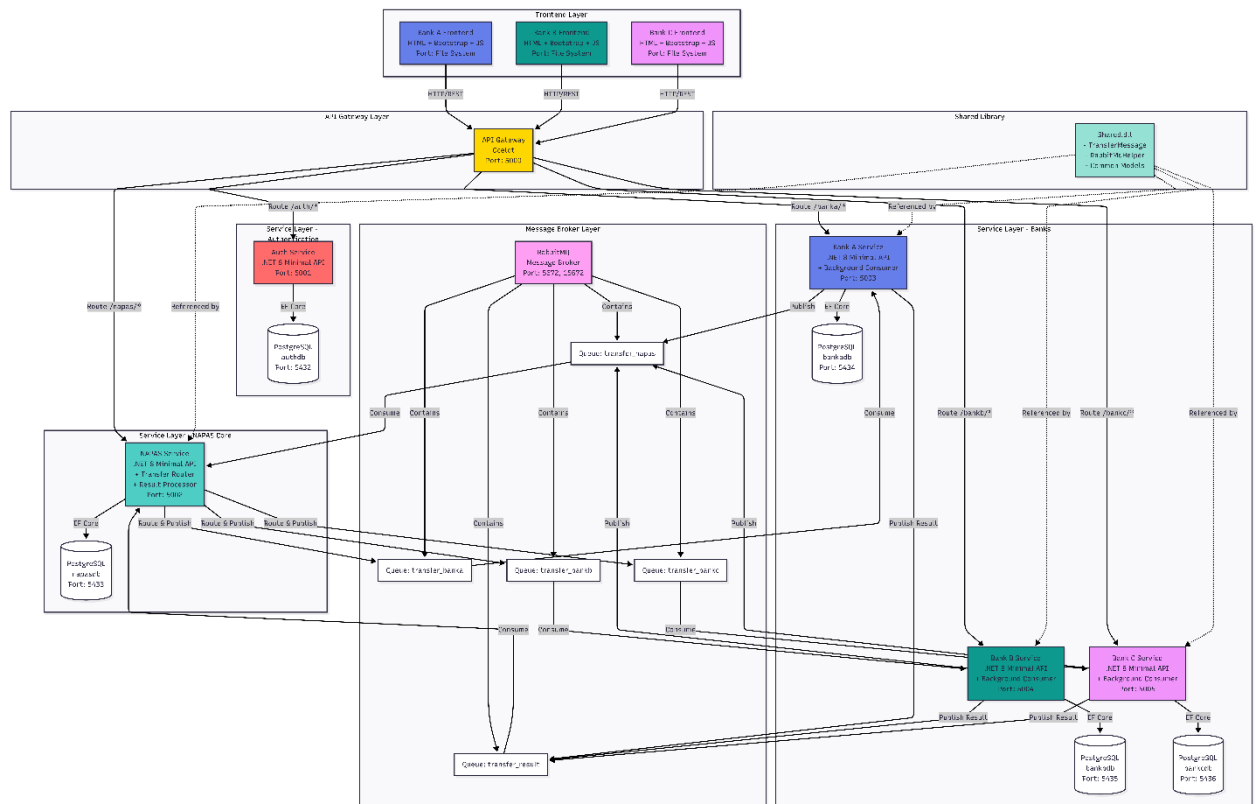
2.8. Demo Data Summary

Entity	Quantity	Details
Users	4	• user_banka (Bank A User) • user_bankb (Bank B User) • user_bankc (Bank C User) • admin (System Admin)
Accounts	9	• Bank A: BANKA001, BANKA002, BANKA003 • Bank B: BANKB001, BANKB002, BANKB003 • Bank C: BANKC001, BANKC002, BANKC003
Initial Balance	-	• Account 001: \$100,000 • Account 002: \$50,000 • Account 003: \$75,000
Transactions	0	Generated during usage

PHẦN III: THIẾT KẾ PHẦN MỀM

1. Thiết Kế Tổng Thể

1.1. Kiến Trúc Layers



Chi tiết từng layer:

1. Presentation Layer:

- **Trách nhiệm:** Giao diện người dùng
- **Technology:** HTML5, JavaScript ES6+, Bootstrap 5, SignalR Client
- **Features:** Login, Dashboard, Transfer forms, Transaction history, Real-time updates

2. API Gateway Layer:

- **Trách nhiệm:** Routing, CORS handling, Single entry point
- **Technology:** Ocelot API Gateway
- **Port:** 5000
- **Routes:** /auth/*, /bank-a/*, /bank-b/*, /bank-c/*, /napas/*

3. Service Layer:

- **Trách nhiệm:** Business logic và data processing
- **Technology:** ASP.NET Core 8.0

- **Pattern:** One service per business domain
- **Services:** Auth (5001), NAPAS (5002), Bank A (5003), Bank B (5004), Bank C (5005)

4. Message Queue Layer:

- **Trách nhiệm:** Asynchronous communication
- **Technology:** RabbitMQ 3.x
- **Ports:** 5672 (AMQP), 15672 (Management UI)
- **Queues:** 5 queues cho inter-bank transfers

5. Data Layer:

- **Trách nhiệm:** Data persistence
- **Technology:** PostgreSQL 15
- **Pattern:** Database per service
- **Databases:** 5 databases độc lập

Supporting Components:

Shared Library:

- **Vai trò:** Cross-cutting component được sử dụng bởi tất cả services
- **Technology:** .NET 8 Class Library (Shared.dll)
- **Components:**
 - **TransferMessage.cs:** DTO cho message queue communication
 - **RabbitMqHelper.cs:** Utilities methods cho RabbitMQ operations
 - **Common Models:** Shared entities, enums (TransferStatus: PENDING, SUCCESS, FAILED)
- **Referenced by:** Auth Service, NAPAS Service, Bank A/B/C Services
- **Lợi ích:**
 - Tránh duplicate code giữa các services
 - Đảm bảo consistency về data contracts
 - Centralized common logic
 - Dễ dàng maintain và update

1.2. Design Patterns Sử Dụng

a. Microservices Pattern

- **Mô tả:** Chia hệ thống thành các services nhỏ, độc lập
- **Lợi ích:**
 - Dễ phát triển và maintain
 - Có thể deploy độc lập
 - Dễ scale từng phần
 - Công nghệ đa dạng (polyglot)

b. Database per Service Pattern

- **Mô tả:** Mỗi service có database riêng
- **Lợi ích:**
 - Tách biệt dữ liệu
 - Tránh coupling giữa services
 - Có thể chọn DB phù hợp cho từng service
- **Trade-off:** Phức tạp trong joins và transactions phân tán

c. API Gateway Pattern

- **Mô tả:** Single entry point cho tất cả services
- **Lợi ích:**
 - Đơn giản hóa client
 - Xử lý cross-cutting concerns (CORS, auth, logging)
 - Load balancing
 - Request aggregation

d. Event-Driven Architecture

- **Mô tả:** Services giao tiếp qua events/messages qua RabbitMQ
- **Implementation:**
 - RabbitMQ (Port 5672, UI: 15672)
 - 5 Queues: transfer_napas, transfer_banka, transfer_bankb, transfer_bankc, transfer_result
 - Message Format: JSON
 - Flow: Bank A → NAPAS → Bank B → NAPAS (update status)

- **Lợi ích:**
 - Loose coupling
 - Asynchronous processing
 - Scalability
 - Resilience
 - Message persistence & retry

e. Repository Pattern

- **Mô tả:** Abstraction layer giữa business logic và data access
- **Lợi ích:**
 - Dễ test
 - Dễ thay đổi database
 - Clean architecture

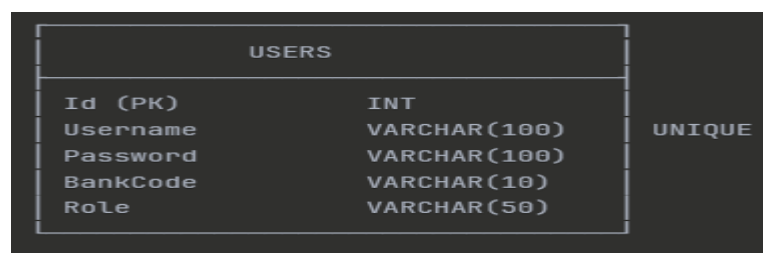
f. Background Service Pattern

- **Mô tả:** Long-running tasks trong background
- **Implementation:**
 - TransferRouterService: Route messages to bank queues
 - ResultProcessorService: Update transaction status & reconciliation
- **Sử dụng:** Message queue consumers
- **Lợi ích:**
 - Non-blocking
 - Parallel processing
 - Better resource utilization

2. Thiết Kế Chi Tiết

2.1. Entity Relationship Diagrams (ERD)

Auth Service Database



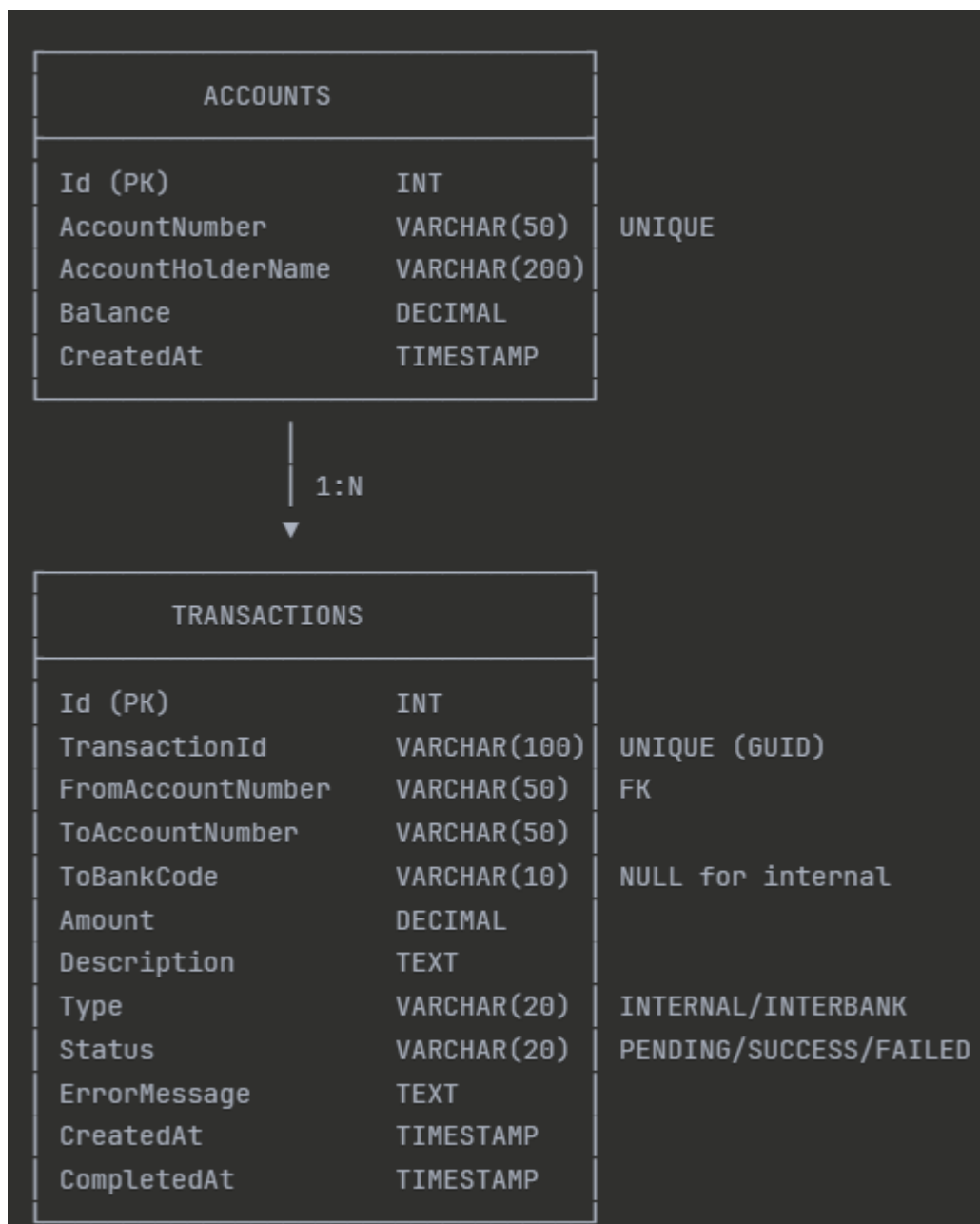
Indexes:

- PRIMARY KEY (Id)
- UNIQUE INDEX (Username)

Sample Data:

- user_banka / pass123 / BANKA / User
- user_bankb / pass123 / BANKB / User
- user_bankc / pass123 / BANKC / User
- admin / admin123 / NAPAS / Admin

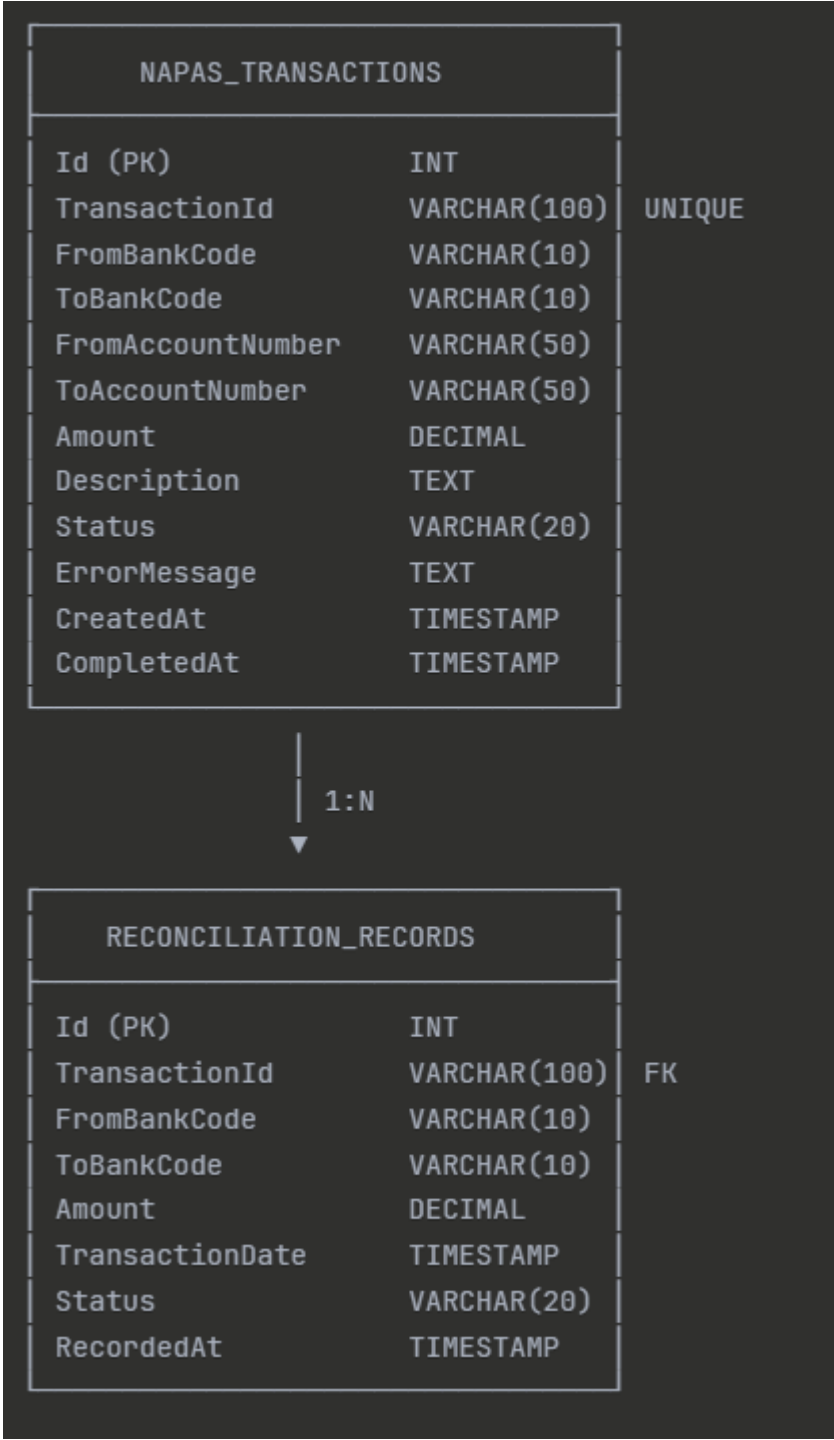
Bank Service Database



Indexes:

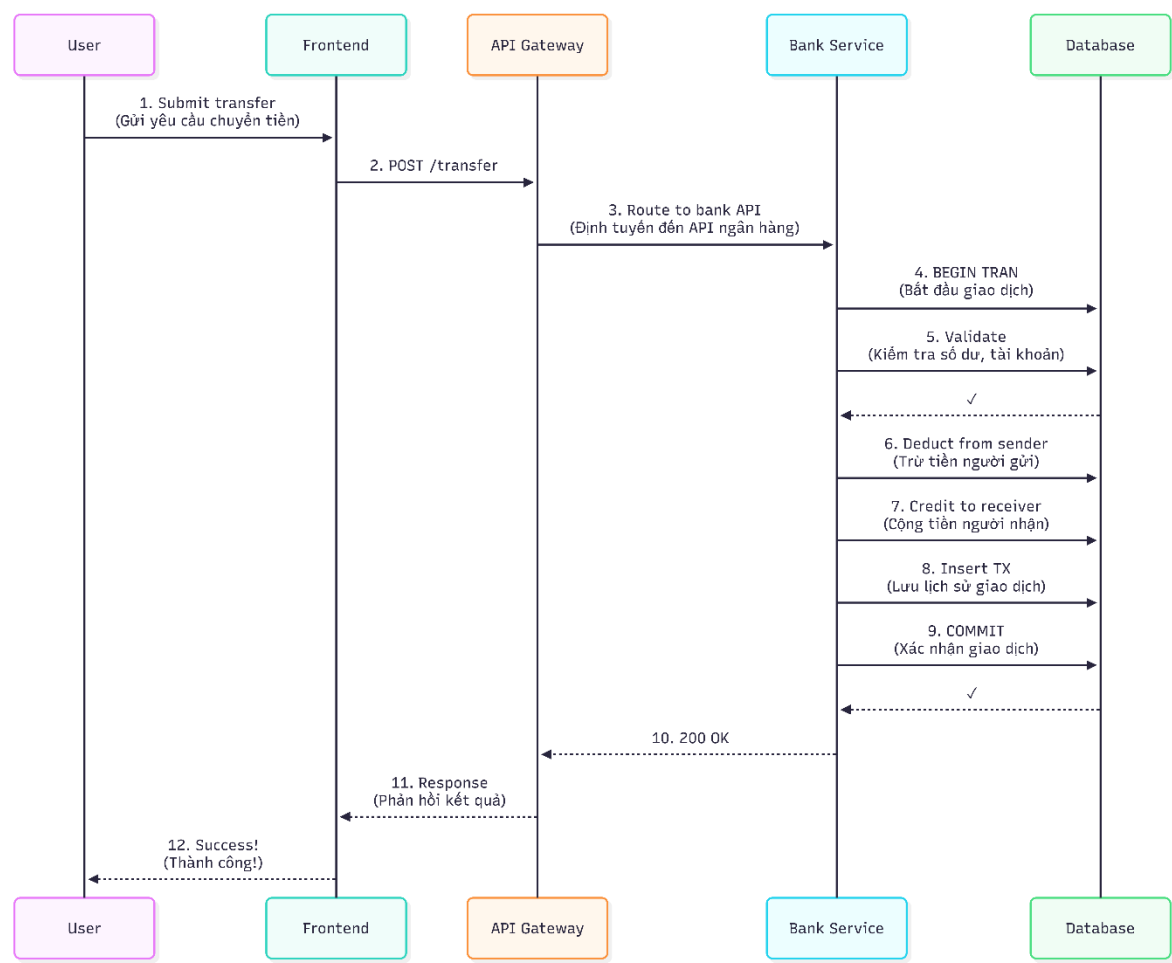
- PRIMARY KEY (Id)
- UNIQUE INDEX (AccountNumber)
- UNIQUE INDEX (TransactionId)
- INDEX (FromAccountNumber)
- INDEX (ToAccountNumber)
- INDEX (Status)
- INDEX (CreatedAt)

NAPAS Service Database



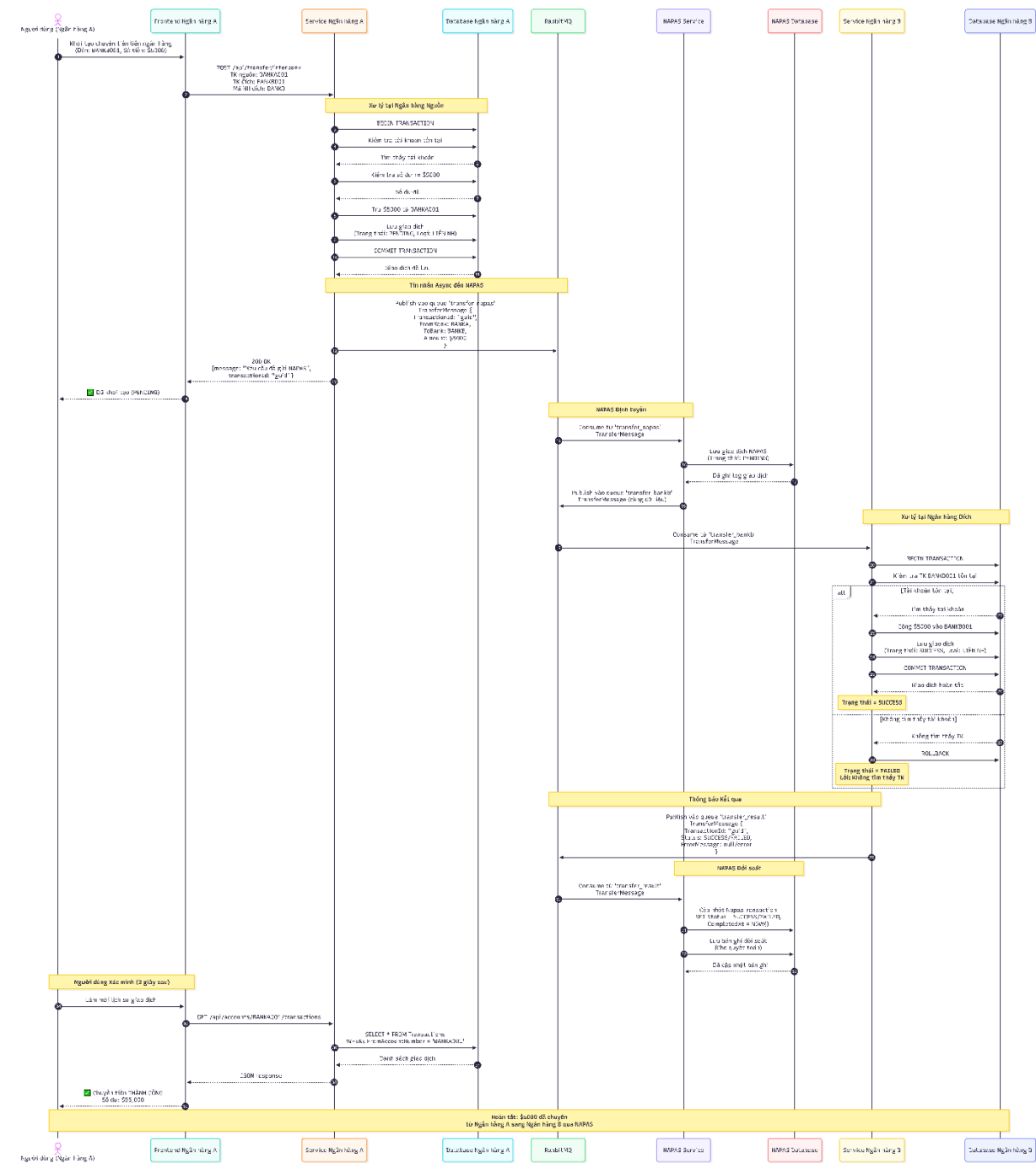
2.2. Sequence Diagrams

Chuyển Tiền Nội Bộ



Thời gian: < 100ms

Chuyển Tiền Liên Ngân Hàng (Chi tiết)



2.3. Class Diagrams

Bank Service Domain Models

```
c#
public class Account
{
    public int Id { get; set; }
    public string AccountNumber { get; set; }
    public string AccountHolderName { get; set; }
    public decimal Balance { get; set; }
    public DateTime CreatedAt { get; set; }
}

public class Transaction
{
    public int Id { get; set; }
    public string TransactionId { get; set; }
    public string FromAccountNumber { get; set; }
    public string ToAccountNumber { get; set; }
    public string? ToBankCode { get; set; }
    public decimal Amount { get; set; }
    public string? Description { get; set; }
    public string Type { get; set; } // INTERNAL, INTERBANK
    public string Status { get; set; } // PENDING, SUCCESS, FAILED
    public string? ErrorMessage { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? CompletedAt { get; set; }
}
```

Shared Models

```
c#
public class TransferMessage
{
    public string TransactionId { get; set; }
    public string FromBankCode { get; set; }
    public string ToBankCode { get; set; }
    public string FromAccountNumber { get; set; }
    public string ToAccountNumber { get; set; }
    public decimal Amount { get; set; }
    public string? Description { get; set; }
    public string Status { get; set; }
    public string? ErrorMessage { get; set; }
    public DateTime Timestamp { get; set; }
}

public enum TransferStatus
{
    PENDING,
    SUCCESS,
    FAILED
}
```

3. Thiết Kế API

3.1. RESTful API Principles

- Sử dụng HTTP methods phù hợp (GET, POST, PUT, DELETE)
- Status codes chuẩn (200, 201, 400, 404, 500)
- JSON format cho request/response
- Resource-based URLs

3.2. API Endpoints Summary

3.2.1. Auth Service (Port 5001)

Authentication Endpoints:

- POST /api/auth/login - Đăng nhập và nhận JWT token
 - Request: { username, password }
 - Response: { token, username, bankCode, role }
- GET /api/auth/users - Lấy danh sách người dùng (Admin only)
 - Response: [{ id, username, bankCode, role }]

Admin Endpoints:

- POST /api/auth/admin/create-tables - Tạo bảng database
- POST /api/auth/admin/seed - Seed dữ liệu mẫu
- POST /api/auth/admin/reset - Reset toàn bộ dữ liệu

3.2.2. Bank Services (Bank A: 5003, Bank B: 5004, Bank C: 5005)

Account Management:

- GET /api/accounts - Lấy danh sách tài khoản của ngân hàng
 - Response: [{ accountNumber, accountHolderName, balance, createdAt }]
- GET /api/accounts/{accountNumber} - Lấy chi tiết tài khoản
 - Response: { accountNumber, accountHolderName, balance, createdAt }
- GET /api/accounts/{accountNumber}/transactions - Lấy lịch sử giao dịch
 - Query params: ?skip=0&take=20&type=ALL|INTERNAL|INTERBANK
 - Response: [{ transactionId, fromAccount, toAccount, amount, type, status, createdAt }]

Transfer Operations:

- POST /api/transfer/internal - Chuyển tiền nội bộ (trong cùng ngân hàng)
 - Request: { fromAccountNumber, toAccountNumber, amount, description }
 - Response: { transactionId, status, message }
 - Thời gian: < 100ms (ACID transaction)
- POST /api/transfer/interbank - Khởi tạo chuyển tiền liên ngân hàng
 - Request: { fromAccountNumber, toAccountNumber, toBankCode, amount, description }
 - Response: { transactionId, status: "PENDING", message }
 - Thời gian: 2-3 giây (async via RabbitMQ)

Admin Operations:

- POST /api/admin/seed - Seed dữ liệu mẫu cho ngân hàng
- POST /api/admin/reset - Reset dữ liệu ngân hàng

Real-time Communication (SignalR):

- WebSocket endpoint: /balanceHub
- Methods:
 - SubscribeToAccount(accountNumber) - Đăng ký nhận cập nhật real-time

- UnsubscribeFromAccount(accountNumber) - Hủy đăng ký
- Events (Server → Client):
 - BalanceUpdated - Thông báo khi số dư thay đổi
 - TransactionReceived - Thông báo khi nhận tiền từ ngân hàng khác
 - TransactionCompleted - Thông báo khi giao dịch hoàn tất

3.2.3. NAPAS Service (Port 5002)

Transaction Management:

- GET /api/transactions - Lấy tất cả giao dịch liên ngân hàng
 - Query
params: ?skip=0&take=20&status=ALL|PENDING|SUCCESS|FAILED
 - Response: [{ transactionId, fromBank, toBank, amount, status, createdAt, completedAt }]
- GET /api/transactions/{transactionId} - Lấy chi tiết giao dịch
 - Response: { transactionId, fromBank, toBank, fromAccount, toAccount, amount, status, errorMessage, createdAt, completedAt }

Reconciliation:

- GET /api/reconciliation - Lấy bản ghi đối soát
 - Query params: ?date=2024-01-01&fromBank=BANKA&toBank=BANKB
 - Response: [{ id, transactionId, fromBank, toBank, amount, transactionDate, status, recordedAt }]

Statistics:

- GET /api/statistics - Lấy thống kê tổng quan
 - Response:

```
json
{
  "totalTransactions": 1250,
  "totalAmount": 5000000,
  "successCount": 1200,
  "failedCount": 50,
  "pendingCount": 0,
  "transactionsByBank": {
    "BANKA": 450,
    "BANKB": 400,
    "BANKC": 400
  }
}
```

Background Services (Internal):

- TransferRouterService - Consume từ queue transfer_napas, định tuyến đến ngân hàng đích
- ResultProcessorService - Consume từ queue transfer_result, cập nhật trạng thái và tạo bản ghi đối soát

3.2.4. API Gateway (Ocelot - Port 5000)

Route Configuration:

- /auth/* → Auth Service (Port 5001)
 - Example: GET http://localhost:5000/auth/api/auth/users
- /bank-a/* → Bank A Service (Port 5003)
 - Example: GET http://localhost:5000/bank-a/api/accounts
- /bank-b/* → Bank B Service (Port 5004)
 - Example: POST http://localhost:5000/bank-b/api/transfer/internal
- /bank-c/* → Bank C Service (Port 5005)
 - Example: GET http://localhost:5000/bank-c/api/accounts/BANKC001
- /napas/* → NAPAS Service (Port 5002)
 - Example: GET http://localhost:5000/napas/api/transactions

Features:

- Điểm vào duy nhất (Single Entry Point) cho tất cả services
- Automatic routing dựa trên path prefix
- CORS handling
- Load balancing (future enhancement)
- Request/Response aggregation (future enhancement)

3.2.5. API Security

Authentication:

- Tất cả endpoints (trừ /api/auth/login) yêu cầu JWT token
- Header format: Authorization: Bearer <token>
- Token expiration: Configurable (mặc định 24h)

Authorization:

- User role: Chỉ truy cập tài khoản thuộc ngân hàng của mình
- Admin role: Truy cập admin endpoints và xem tất cả dữ liệu

- BankCode validation: User không thể truy cập tài khoản của ngân hàng khác

Error Responses:

- 401 Unauthorized - Missing hoặc invalid JWT token
- 403 Forbidden - Không có quyền truy cập resource
- 404 Not Found - Resource không tồn tại
- 400 Bad Request - Invalid request data
- 500 Internal Server Error - Server error

Standard Response Format:

```
json
{
  "success": true/false,
  "message": "Description",
  "data": { ... },
  "error": "Error message if failed"
}
```

4. Thiết Kế Message Queue

4.1. Queue Configuration

- **Durable:** true (queues survive broker restart)
- **Auto-delete:** false
- **Exclusive:** false
- **Message TTL:** không giới hạn (cho demo)

4.2. Message Flow

1. Bank Service publish TransferMessage → transfer_napas
2. NAPAS consume từ transfer_napas
3. NAPAS publish TransferMessage → transfer_{bankcode}
4. Destination Bank consume từ transfer_{bankcode}
5. Destination Bank publish Result → transfer_result
6. NAPAS consume từ transfer_result

4.3. Error Handling

- Failed messages không bị mất (acknowledgment mechanism)
- Có thể implement Dead Letter Queue (DLQ)

- Retry logic trong consumer
- Idempotent processing (dùng TransactionId)

5. Thiết Kế Security

5.1. Authentication (JWT)

- Algorithm: HS256
- Claims: Username, BankCode, Role
- Expiration: Configurable
- Secret key: Từ environment variable

5.2. Authorization

- Kiểm tra JWT token cho mọi protected endpoint
- Middleware authentication trong ASP.NET Core
- User chỉ truy cập tài khoản thuộc ngân hàng của mình

5.3. Data Protection

- Database isolation (không shared database)
- No direct database access giữa services
- Sensitive data (password) nên hash (TODO cho production)

6. Thiết Kế Deployment

6.1. Containerization

- **Docker:** Mỗi service là một container
- **Base Image:** Microsoft .NET SDK (build) + ASP.NET Runtime (run)
- **Multi-stage builds:** Giảm image size

6.2. Orchestration

- **Docker Compose:** Development và demo
- **Kubernetes:** Production (future)

6.3. Network

- **Custom bridge network:** napas-network
- Service discovery qua container names
- Port mapping: 5000-5005, 5432-5436, 5672, 15672, 8080

6.4. Volumes

- **Database volumes:** Persistent data storage
- **Named volumes:** postgres-data- {service}